Artificial Intelligence Reasoning Project II

Arsh Beg
Faculty of Informatics
Université Toulouse Capitole
Toulouse, France
arsh.beg@ut-captiole.fr

Christina Berghegger
Faculty of Informatics
Université Toulouse Capitole
Toulouse, France
christina.berghegger@ut-capitole.fr

Thi Bich Pham
Faculty of Informatics
Université Toulouse Capitole
Toulouse, France
thi-bich.pham@ut-capitole.fr

Esrom F. Tebebu
Faculty of Informatics
Université Toulouse Capitole
Toulouse, France
esrom.tebebu@ut-capitole.fr

***Abstract*—This report provides an analysis of four algorithms for finding a solution for a rugby tournament: Depth-First Search (DFS), Breadth-First Search (BFS), A* (A-star), and CSP.**

I. Introduction

The six nations tournament poses a challenging scheduling issue that must be followed in order to maintain excitement and fairness. Organisers have to restrict teams from playing back-to-back home or away games and restrict teams from playing the same opponents more then once. Particular matches also need to be accommodated, like England vs France on the last day. We attempt to model and solve the six nations scheduling challenge in this research. Our goals are to assess how long such a work would take. Our method entails creating multiple search algorithm in python and modelling the problem as both a conventional search and a constraint satisfaction problem. We aim to find the most effective ways to generate tournament fixtures within the given limits by means of experimentation.

II. Modelling

A. Standard Search Problem

To approach the scheduling problem in terms of a standard search problem, a tree needs to be initialised. After, the trees branches can be iteratively analysed, nodes being activated through functions and paths being evaluated based on constraints and goal tests. The properties of the six nations tournament scheduling problem expressed as a standard search problem are as follows:

- **Depth:** The tree needs to have a depth of five, each layer representing one match round. Since the tournament is based on a mirroring theme, it is sufficient to schedule only the first five rounds, which can then be mirrored to obtain the entire schedule over ten rounds. This will significantly reduce efforts in terms of time and memory used to find a solution.
- **Nodes:** Each node in the tree represents one round, expressed as a triplet of tuples, each tuple containing two teams that play against each other. The order within the tuple is important, the first team being the one playing at home, the second team being the one playing away.

$$N = ((Team_1, Team_2), (Team_3, Team_4), (Team_5, Team_6)) \quad (1)$$

- **Initial state:** The tree is initialised as an empty tournament.
- **State:** A state can be described as the combination of teams that play against each other, the order of teams within the tuples representing a match being relevant.
- **Action**: To explore new paths and nodes in the tree, the defined action can be described as adding a new round in terms of combination of matches having no conflict to reach the goal tests. Conflicts are natural constraints that the tournament needs to have for a functioning setup. Natural constraints are assumed as follows:
  - No team plays with itself in a match
  - No team play twice in a round
  - No same combination of teams in terms of a tuple appears more than once in the entire five rounds, the order being irrelevant

  Taking these natural constraints into account while building the tree considerably reduced the search space, which has an important impact on execution time and memory usage when conducting the search. Since natural constraints already rule out numerous child nodes, the **branching factor** of the tree is 120, each node representing one possible combination of three matches. Without applying the action in terms of natural constraints, the branching factor would increase to 3640. Hence, expanding and exploring the tree through the application of the defined action is crucial for performance reasons.
- **Goal state**: The goal is a valid tournament with five rounds, composed of three matches each, that passes all constraints. Furthermore, the goal state can have additional constraints based on scheduling requirements, that can be checked when having reached a node of depth five. In this project, the following additional constraints are explored:
  - No two teams can play twice at home in consecutive rounds, or twice away in consecutive rounds
  - No team plays against ITALY or FRANCE away twice in consecutive rounds
  - In the last round ENGLAND plays against FRANCE

  Please note that one of the constraints requested by the organizers of the tournament could not be implemented: No team can play home or away twice in a row. With this constraint, a valid solution cannot be found.

B. Heuristic Search with A*

For the A* star algorithm, an acceptable heuristic needs to be defined. For this purpose, the heuristic that was found to be acceptable is one that allocates a value to each node based on the number of "conflicts" caused by asymmetric match placements per node. To calculate it, where N is a node representing a state (in this case a match (T_α, T_β) where T_α is playing at home against T_β which is playing away $\forall\alpha, \beta \in T, (\alpha \neq \beta)$), $h(N)$ is the number of conflicts for a given node. It is the sum of the number of conflicts caused by F which evaluates if the final match constraint—England, France play against each other—is not violated, $S(N_X, N_Y)$ evaluates if the same team is not playing more than once for a given round $A(N_X, N_Y)$ evaluates if the match placements are asymmetrical (i.e. there exists more than one match where two teams play against each other), and $L(N_X, N_Y)$ evaluates if a team is not playing away against France or Italy in consecutive rounds.

$$h(N) = \sum_{i=0}^{N} F(N_i) \sum_{j=0}^{N} A(N_i, N_j) + S(N_i, N_j) + L(N_i, N_j)$$

where

for the final match constraint $i \in \{0, ..., |N| - 0.5 \cdot |T| - 1\}$,

$$F(N_i) = \begin{cases} 1 & \text{if } \exists t \in N_i, all(England, France) \in t \\ 0 & \text{else} \end{cases}$$

for the symmetry constraint $i \neq j$,

$$A(N_i, N_j) = \begin{cases} 1 & \text{if } \forall t \in N_i, \nexists t \notin N_j \\ 0 & \text{else} \end{cases}$$

for the scheduling constraint $i \neq j, \lfloor \frac{i}{0.5 \cdot |T|} \rfloor = \lfloor \frac{j}{0.5 \cdot |T|} \rfloor$

$$S(N_i, N_j) = \begin{cases} 1 & \text{if } \forall t \in N_i, \exists t \in N_j \\ 0 & \text{else} \end{cases}$$

for the reduced location constraint,

let t_0 = France and t_1 = Italy

if $(\forall t = N_{i,home} \wedge t = N_{j,away})$
$\wedge (\lfloor \frac{i}{0.5 \cdot |T|} \rfloor = 0 \wedge \lfloor \frac{j}{0.5 \cdot |T|} \rfloor = 0.5 \cdot |T|)$

$$L(N_i, N_j) = \begin{cases} 1 & \text{if } N_{i,home}, N_{j,home} = (t_0 \vee t_1) \\ 0 & \text{else} \end{cases}$$

else $(\forall t = N_{i,home} \wedge t = N_{j,home})$
$\wedge (\lfloor \frac{i}{0.5 \cdot |T|} \rfloor = \lfloor \frac{j}{0.5 \cdot |T|} + 1 \rfloor)$

$$L(N_i, N_j) = \begin{cases} 1 & \text{if } N_{i,away}, N_{j,home} = (t_0 \vee t_1) \\ 0 & \text{else} \end{cases}$$

To implement this heuristic using the open-source code from (Russel, Norvig, 2010) [1], Algorithm 1 will give the number of conflicts per node.

Algorithm 1 Heuristic(Node)

```
|C| ← 0                                   ▷ |C| is the number of conflicts
for i, N_i in Node.State do
    if i ∈ {0, ..., |N_i| − 0.5 · |T| − 1} then
        |C| ← |C| + F(N_i)                ▷ Final match constraint violation(s)
    end if
    for j, N_j in Node.State do
        if i ≠ j then
            |C| ← |C| + A(N_i, N_j)       ▷ Symmetry constraint violation(s)
        end if
        if i ≠ j ∧ ⌊i/(0.5·|T|)⌋ = ⌊j/(0.5·|T|)⌋ then
            |C| ← |C| + S(N_i, N_j)       ▷ Scheduling constraint violation(s)
        end if
        if [(t = N_{i,home} ∧ t = N_{j,away}) ∧ (⌊i/(0.5·|T|)⌋ = 0 ∧
        ⌊j/(0.5·|T|)⌋ = 0.5 · |T|)] ∨ [t = N_{i,home} ∧ t = N_{j,home}) ∧
        (⌊i/(0.5·|T|)⌋ = ⌊j/(0.5·|T|) + 1⌋)] then
            |C| ← |C| + L(N_i, N_j)       ▷ Reduced location constraint violation(s)
        end if
    end for
end for
return |C|
```

C. Constraint Search Problem (CSP)

Scheduling tasks are a famous example of problems where Constraint Search can be applied. Constraint Search Problems model a problem in terms of constraint satisfaction, the constraints being defined in terms of admissible values that a set of variables of a problem can have. A solution is only valid if all variables have an admissible value, consequently do not breach any constraint. In the process of searching a solution, the CSP takes on different states. A state is an assignment of a value to one or more variables, whose admissibility can then be checked by checking all constraints the variable has with other variables. One of the main advantages of CSPs are their quick elimination of search branches through iterative constraint validation. Widely used approaches for the solution search in CSPs are backtracking and forward checking. In backtracking, a partial consistent solution is incrementally extended towards a complete solution by checking values to a new variable for conflicts. If a checked value violates a constraint with the partial consistent solution, backtracking goes back to the last instantiated variable that still has available alternatives and the subspace of the violating variable can be eliminated. In forward checking, constraints are checked between current variables and neighboring variables. The domain of neighboring variables are filtered for values that are not in a constraint with the current variable. If one of the future domains is empty because all values violate constraints with the current solution, the branch is found to be non-compatible with the current solution and can be immediately eliminated,

which is why forward checking allows to prune trees earlier than backward checking, being one of its main advantages [2]. Furthermore, forward checking allows for the optimization of the search through two widely used heuristics, amongst others. The first heuristic is concerned with choosing the next variable, preferring the variable with the smallest filtered domain to expand the tree, in other words the variable with the most constraints. The second one is concerned with the value this variable should take one, preferring the value which has the least constraints.

III. Implementation

In the following section, the implementation of the different search approaches as explained above will be outlined. First, the standard search problem in terms of breath-first search (BFS) as well as depth-first search (DFS) will be explored, then the implementation of the constraint satisfaction problem (CSP) will be detailed.

A. Implementation of Standard Search Problem

To prepare for searching by using algorithms DFS and BFS, we started with generating all possible matches, which are pairings between a list of teams. Using nested loops, it iterates through each team in the list, comparing it against every other team to ensure all possibilities. It contains a condition to ignore such cases to avoid duplicative pairings where a team plays against itself. A tuple containing the two teams involved is appended to the list for each valid pairing, and then a list will be returned. Constraints are crucial in solution search functions as they simplify graph exploration because they define the search space by outlining limits and setting conditions for solutions. After achieving all possible matches, we defined the constraints as goal tests, including natural and additional ones.

- **Goal test 1**: Each round, all 6 teams of all matches must be different (natural)
- **Goal test 2**: There can be no duplicate combination of teams considering all matches in a round (natural)
- **Goal test 3**: The last round must contain one match between FRANCE and ENGLAND, the order does not matter
- **Goal test 4**: No team plays against ITALY or FRANCE away twice in consecutive rounds
- **Goal test 5**: No two teams can play twice at home in consecutive rounds, or twice away in consecutive rounds

The natural constraints (1 and 2) will be implemented directly into graph generating to reduce the complexity of execution and the goal test 5 will be automatically eliminated due to its impossibility. We first create a dictionary graph for 1 round by recursively generating all match combinations based on all possible matches, current rounds, and teams. It iterates through each match in the input list, forming new rounds by adding valid matches to the current variable until its length equals half the teams' length. Once a complete round is done, it checks for uniqueness and adds it to the list of rounds. This process continues recursively until all match combinations are finished, and the function returns the list of generated combinations represented as a dictionary. Then, a tree for the half tournament, where each node represents a round, and the depth parameter defines the depth of the tree, is created based on the graph of the rounds. It assigns the round to each node. Then, it iteratively grows the tree by duplicating each layer's existing structure. Once the expected depth is reached, the function produces the tree structure.

Algorithm 2 generate_graph(matches, current_round, rounds, teams)

if len($current_round$) = $\frac{\text{len}(teams)}{2}$ **then**
  $combined_matches \leftarrow ()$
  for tpl **in** $current_round$ **do**
    $combined_matches \leftarrow combined_matches + (tpl,)$
  end for
  if CHECK_UNIQUE_ORDER($rounds, combined_matches$) **then**
    rounds.append($combined_matches$)
  end if
  return
end if
for i **in** range(len($matches$)) **do**
  $match \leftarrow matches[i]$
  $new_matches \leftarrow []$
  for m **in** matches$[i:]$ **do**
    if CHECK_MIRROR($[match, new_matches], m$) **and** CHECK_EXISTED($[match, new_matches], m$) **then**
      new_matches.append(m)
    end if
  end for
  generate_graph($new_matches, current_round + [match], rounds, teams$)
end for
return $rounds$

DFS implementation - Algorithm 3: Our DFS implementation aims to traverse following the tree's depth with the idea of branch by branch. An action is defined as adding a valid round to a solution, and as we discussed before, each node is a combination of matches (represented as a round). The algorithm begins at a specified node, which is at the first layer of the graph. It traverses through the graph, exploring each branch as deeply as possible before backtracking. When we reach an invalid node, we skip all its children nodes to reduce time execution. This process continues until a solution is found or all possible paths have been explored. The 'constraints' parameter allows for additional constraints when reaching the node at the last layer during the search process, altering the algorithm's behavior accordingly. Throughout the search, the algorithm keeps track of the current layer or depth and the path to that point. **BFS implementation - Algorithm 4**: Our BFS algorithm examines paths within a graph structure with the idea of layer by layer, which traverses almost the whole graph to find the solution. The algorithm iterates through each layer

Algorithm 3 DFS(node, constraints, layer, path, graph, teams)

if $layer <$ len$(teams) - 1$ **then**
  $path[layer] \leftarrow node$
  if not GOAL_TEST_CONDITION_2($path[: layer + 1]$) **then**
    return None
  end if
  $next_nodes \leftarrow$ graph$[node]$
  if $layer =$ len$(teams) - 2$ **then**
    $next_nodes \leftarrow$ graph
    if $constraints$ **then**
      if GOAL_TEST_CONDITION_4($path$) **and** GOAL_TEST_CONDITION_3($path$) **then**
        return $path$
      end if
    else
      return $path$
    end if
  end if
  for $next_node$ **in** next_nodes.keys() **do**
    $result \leftarrow$ DFS$(next_node, constraints, layer + 1, path, next_nodes, teams)$
    if $result \neq$ **None then**
      return $result$
    end if
  end for
end if
return None

Algorithm 4 bfs(graph_bfs)

$layers \leftarrow 0$
$current_layer \leftarrow [(graph_bfs, ['root'])]$
while $current_layer$ **do**
  $next_layer \leftarrow []$
  for $node_dict, path$ **in** $current_layer$ **do**
    for $key, value$ **in** node_dict.items() **do**
      if isinstance($value$, **dict**) **then**
        if $layers > 0$ **then**
          if (GOAL_TEST_CONDITION_2($path[1 :] + [key]$) **and** GOAL_TEST_CONDITION_4($path[1 :] + [key]$)) **then**
            $next_layer$.append($(value, path + [key])$)
          end if
        else
          $next_layer$.append($(value, path + [key])$)
        end if
      else
        for v **in** $value$ **do**
          if GOAL_TEST_CONDITION_2($path[1 :] + [v]$) **and** GOAL_TEST_CONDITION_4($path[1 :] + [v]$) **then**
            if GOAL_TEST_CONDITION_3($path[1 :] + [v]$) **then**
              return $path[1 :] + [v]$
            end if
          end if
        end for
      end if
    end for
  end for
  $current_layer \leftarrow next_layer$
  $layers \leftarrow layers + 1$
end while
return "No solution"

of nodes, exploring all possible nodes at each layer. All valid nodes at one specific layer will be added to a list 'next_layer' for the subsequent tracking with the path to the root. Then, at the final step, we have a list of potential solutions that need to be validated with additional constraints. This process continues until a solution meets detailed conditions and all layers have been explored. The algorithm efficiently searches through the graph, ensuring each layer is explored before moving to the next one.

B. Implementation of CSP

The initial steps to implement a CSP are the variables, domains and constraints. Variables are a complete set of variables relevant to the problem, domains are defined in terms of admissible values variables can take on, and constraints can be considered to be a set of rules regarding admissibility of values, both in terms of single variables and combination of variables. For the six nations tournament, variables, domains and constraints can be defined as follows. For the implementation, the library python-constraint was used.

1) Variables: In total, the six nations tournament has a set of 30 variables, each variable representing one team per tournament round. As described above, it is sufficient to implement the tournament with five instead of ten rounds, given it is based on a mirroring scheme which can be applied to the second half of the tournament. Therefore, it can be assumed the tournament has five rounds in total, each round being composed of three matches, each match being composed of two teams. Hence, six teams per round over five rounds, totals 30 variables. In this approach, the order of the variables matter (as indicated by the variable names indicating the round and team number), assuming that matches are played per round between team one and two, team three and four, and team five and six.

$$
\begin{aligned}
X = \{&X_{r1t1}, X_{r1t2}, X_{r1t3}, X_{r1t4}, X_{r1t5}, X_{r1t6},\\
&X_{r2t1}, X_{r2t2}, X_{r2t3}, X_{r2t4}, X_{r2t5}, X_{r2t6},\\
&X_{r3t1}, X_{r3t2}, X_{r3t3}, X_{r3t4}, X_{r3t5}, X_{r3t6},\\
&X_{r4t1}, X_{r4t2}, X_{r4t3}, X_{r4t4}, X_{r4t5}, X_{r4t6},\\
&X_{r5t1}, X_{r5t2}, X_{r5t3}, X_{r5t4}, X_{r5t5}, X_{r5t6}, \}
\end{aligned} \tag{2}
$$

2) Domains: In the six nations tournament, the domain of each variable is equal to the teams participating in the tournament. If teams are defined in terms of names, the domain for each of the 30 variable is as follows.

$$D = \{'ENGLAND','FRANCE','ITALY', \\ 'WALES','SCOTLAND','IRELAND'\} \tag{3}$$

3) Constraints: The constraints of the six nations tournament can be divided into natural constraints and additional constraints. Natural constraints refer to constraints necessary for a realistic implementation of the tournament, while additional constraints are rules than can be applied to the scheduling, but are not necessarily needed for a functioning setup.

Natural constraint 1: All six teams need to play in each round.

$$\begin{aligned} C_1 &= \{Alldiff(X_{r1t1}, X_{r1t2}, X_{r1t3}, X_{r1t4}, X_{r1t5}, X_{r1t6})\} \\ C_2 &= \{Alldiff(X_{r2t1}, X_{r2t2}, X_{r2t3}, X_{r2t4}, X_{r2t5}, X_{r2t6})\} \\ C_3 &= \{Alldiff(X_{r3t1}, X_{r3t2}, X_{r3t3}, X_{r3t4}, X_{r3t5}, X_{r3t6})\} \\ C_4 &= \{Alldiff(X_{r4t1}, X_{r4t2}, X_{r4t3}, X_{r4t4}, X_{r4t5}, X_{r4t6})\} \\ C_5 &= \{Alldiff(X_{r5t1}, X_{r5t2}, X_{r5t3}, X_{r5t4}, X_{r5t5}, X_{r5t6})\} \end{aligned} \tag{4}$$

Natural constraint 2: No teams play against each other twice during the first five rounds of the tournament.

This constraint needs to be defined for each tuple of teams (assuming the grouping of teams as explained above) in comparison with all other tournaments of the entire tournament. In the following, the constraint will be explained with the example of the tuple $X = \{X_{r1t1}, X_{r1t2}\}$ (round one match 1) in comparison with the first match of round two $X = \{X_{r2t1}, X_{r2t2}\}$, but needs to be implemented for all combinations of tuple of teams (equal to one match) of the entire tournament.

$$\begin{aligned} C_6 = <&\{(X_{r1t1}, X_{r1t2}), (X_{r2t1}, X_{r2t2})\}, \\ &\{((A,B),(C,D)), ((B,A),(C,D)), \\ &((A,B),(D,C)), ((B,A),(D,C))\} > \end{aligned} \tag{5}$$

Additional constraint 1: England and France need to play against each other in the last round.

$$\begin{aligned} C_7 = <&\{(X_{r3t1}, X_{r3t2}), (X_{r3t1}, X_{r3t2}), (X_{r3t5}, X_{r3t6})\}, \\ &\{((A,B),(D_{r3t3}, D_{r3t4}), (D_{r3t5}, D_{r3t6})), \\ &((D_{r3t1}, D_{r3t2}), (A,B), (D_{r3t5}, D_{r3t6})), \\ &((D_{r3t1}, D_{r3t2}), (D_{r3t3}, D_{r3t4}), (A,B))\} > \\ \\ &(A,B) \in \{('FRANCE','ENGLAND') \\ &('ENGLAND','FRANCE')\} \end{aligned} \tag{6}$$

Additional constraint 2: No team plays against Italy or France twice away in two consecutive rounds.

This rather complex constraints was implemented as a constraint function, allowing for a more sophisticated rule set than the constraints explained above. The pseudocode of the function is as follow:

Algorithm 5 Additional Constraint 2

input: list of round in terms of five triplets, each containing three matches in terms of tuples

```
opponentsCurrent ← []
for triplet in input do
    opponentsNew ← []
    for match in triplet do
        if match[0] in ('ITALY','FRANCE') then
            if match[1] in opponentsCurrent then
                return None
            end if
            opponentsNew.append(match[1])
        end if
    end for
    opponentsCurrent ← opponentsNew
end for
return None
```

To solve the CSP, the library proposed different solvers extending a base solver. Specific solvers for backtracking and forward checking, as described in section II, can be used. However, after checking performance of different solvers, the base solver proved to have the best performance in finding a solution to our problem.

C. Implementation of A^* Search

To implement the A* algorithm, we used the greedy best first graph search (GBFS) algorithm from the (Russel, Norvig, 2010) [1] open-source code as shown in Algorithm 6. GBFS uses the heuristic function and its frontier is a priority queue determined by the heuristic value of the node; the algorithm chooses the path with the smallest heuristic value.

A^* Search, uses the heuristic value, and the "naïve" path cost—in this case, each step increments the path cost by one—and its implementation is similar to that of the GBFS. Then, the A* algorithm is implemented as shown in Algorithm 7

IV. Results

After running the DFS, BFS, A*, and CSP, we can compare their average execution times as done in Table I. *Nota bene*: A* and CSP were implemented on different code.

Comparing the depth-first and breadth-first search approaches, the difference in execution time depending on the number of teams is considerable. While both DFS and BFS perform very well on a tournament with four teams and execution time is almost the same, BFS performs considerably worse for a tournament with six teams. This can be explained by the fact that the solution of the scheduling problem

Algorithm 6 BFGS(Problem, f)

```
f ← memoize(f)
n ← Node(Problem.Initial-State)
Frontier ← PriorityQueue('min', f)
Frontier ← Add(n, Frontier)
Explored ← {}
while Frontier do
    n ← Pop(Frontier)
    if Problem.Goal-Test(n.State) then
        Explored ← Add(n.State, Explored)
    end if
    for Child in n.Expand(Problem) do
        if Child.State not in Explored and Child not in
Frontier then
            Frontier ← Add(Child, Frontier)
        else if Child in Frontier then
            if f(Child) < Frontier[Child] then
                Frontier ← Remove(Child, Frontier)
                Frontier ← Add(Child, Frontier)
            end if
        end if
    end for
end while
return None
```

Algorithm 7 A^*(Problem)

```
1: Heuristic-Cost ← memoize(Problem.Heuristic-Cost)
2: return BFGS(Problem, n.Path-Cost + Heuristic-Cost(n))
   for n in Problem.State
```

necessarily needs to be found in the deepest layer of the tree. Hence, when looking for only one solution, it is more effective to check each path until the very end (DFS), and not analyse entire layers before moving onto the next one (BFS). If all solutions to the scheduling problem were to be found, it is likely that DFS and BFS would have a similar performance in execution time. Furthermore, to test the applicability of the search approaches to more complex tournaments, they were tested with a tournament setup containing eight teams. Both approaches did not deliver solutions in an acceptable amount of time and are thus not recommended to be used for tournaments exceeding six teams.

The A^* Search algorithm with a simple—admissible—heuristic and *naïve* cost path proves to be "optimal." Although it has the same worst case execution time as BFS (i.e. $O(n) = b^d$), the A^* search algorithm implementation proved to be significantly faster than BFS. However, it was slower than DFS (which has a linear worst case execution time: $O(n) = b \cdot d$).

The constraint search problem (CSP) performed substantially better with more complex tournament setups. The increase in execution time when adding more teams to the tournament is negligible, turning CSP into a very suitable approach for more complex scheduling tasks.

The approaches presented above all proved to be able to find solutions to the scheduling problem of the six nations tournament, some of them being better than others in terms of execution time and performance. However, the solution cannot be described as optimal. For example, by the way that the graph is created in this project, and without optimization, it is very likely the solution will contain at least one team playing all matches at home. This is because the search algorithms do not take into account how many times teams play at home, and how many times they play away. In real life, this tournament setup is not desirable. Therefore, an optimization in terms of match locations is recommended. This could be achieved with different approaches, such as balancing the number of home and away matches per team, or by the extension of the scheduling tasks with match locations in terms of stadiums. Opting for this approach, the optimization can be defined as constrains where teams have to play in a defined number of stadiums a minimum number of times. This optimization was outside the scope of this project and shall be explored in the future.

TABLE I
AVERAGE EXECUTION TIME (IN SECONDS)

Problem	DFS	BFS	A*	CSP
4 Teams without constraints	0.00004	0,00026	0.0024	0.001421
6 Teams without constraints	0,001	182,9	0.0276	0.0047
8 Teams without constraints	xx	xx	0.2253	0.019671
4 Teams with constraints	0,0002	0,0017	0.0032	0.000879
6 Teams with constraints	0,012	154,66	0.0829	0.004939
8 Teams with constraints	xx	xx	0.5175	0.023754

V. CONCLUSION

In this study, we addressed the problem of scheduling the six nations rugby tournament. Our goal was to evaluate how well different search algorithms and constraint satisfaction solvers generated usable schedule that compiled with the given constraints. We define the problem as a normal search problem as well as constraint satisfaction problem (CSP) through modelling and implementation.

During the solving process, we developed Breadth first search (BFS), Depth first search (DFS), CSP and A* using Python. By testing on smaller cases both with and without constraints, we evaluated the performance of each approach. Our results, which are summarized in a table, clarified how long each algorithm took to run across different scenarios. By our findings we can say that CSP proved to be a better solution for problems like scheduling even with larger tournament with more number of constraints.

At last, the scheduling problem of Six Nations tournament serves as an excellent example of how complexity and strategic decision-making interact. We can overcome these challenges by utilising computational and algorithmic approach.

REFERENCES

[1] Russell, S., Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*. Prentice Hall.

[2] Roman Barták 1998, *Guide to Constraint Programming*, accessed 17 April 2024, ¡https://ktiml.mff.cuni.cz/ bartak/constraints/propagation.htmlFC/¿